

Cloud Foundation Toolkit

來源：<https://codelabs.developers.google.com/cft-onboarding?hl=zh-tw>

步驟數：12

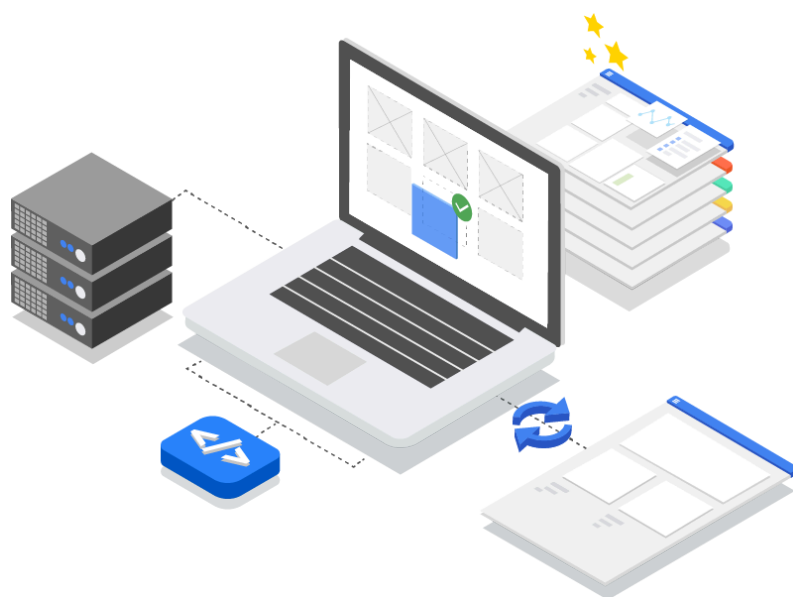
在這個程式碼研究室中，您可開始使用 Cloud Foundation Toolkit(CFT)，並逐步引導您在 CFT 模組中新增功能。

目錄

1. CFT 101 簡介
2. 設定開發環境
3. 為 CFT 的 GCS 存放區建立分支
4. 建立測試環境
5. 在 CFT 模組中新增功能
6. 在儲存空間 bucket 範例中新增功能
7. 更新藍圖測試以檢查功能
8. 在 CFT 中執行整合測試
9. 生成輸入和輸出內容的說明文件
10. 在 CFT 中執行 Lint 測試
11. 在 GitHub 上提交提取要求
12. 恭喜

步驟 1 · 約 0 分鐘

1. CFT 101 簡介



上次更新時間：2022 年 2 月 11 日

什麼是 Cloud Foundation Toolkit ？

基本上，CFT 提供最佳做法的**範本**，讓使用者能夠快速上手 Google Cloud Platform。在本教學課程中，您將瞭解如何貢獻 Cloud Foundation Toolkit。

軟硬體需求

- GitHub 帳戶。
- 在電腦上安裝 Docker，或使用 Cloud Shell ([Mac 安裝](#)、[Windows 安裝](#))
- 程式碼編輯器，用於編輯程式碼 (例如 [Visual Studio Code](#))
- 對 Git 和 GitHub 有基本瞭解
- 具備 Terraform 和基礎架構即程式碼的經驗
- 將「專案建立者」角色授予服務帳戶的權限
- Google Cloud 機構、測試資料夾和帳單帳戶

建構項目

在本程式碼研究室中，您將瞭解如何為 Cloud Foundation Toolkit (CFT) 貢獻心力。

您將學會以下內容：

- 設定開發環境，以便為 CFT 貢獻心力
- 在 CFT 模組中新增功能
- 為新增功能新增測試
- 在 CFT 中執行整合測試
- 執行 Lint 測試
- 將程式碼提交至 GitHub，並提交提取要求 (PR)

您將透過在 [Google Cloud Storage CFT 模組](#) 中新增新功能，執行上述所有步驟。您將新增名為 `"silly_label"` 的標籤，系統會自動將這個標籤新增至透過 GCS CFT 模組建立的所有值區。您也會撰寫測試，驗證功能並確保端對端整合。

2. 設定開發環境

如要開發，可以使用 Cloud Shell。如果您不想使用 Cloud Shell 參與 CFT，可以在電腦上設定開發環境。

設定 Git

GitHub 是以名為 Git 的開放原始碼版本管控系統 (VCS) 為基礎。Git 負責處理本機電腦或 Cloud Shell 上所有與 GitHub 相關的事項。

1. 使用 Cloud Shell 時，不需要安裝 git，因為系統已預先安裝。



```
$ git --version
# This will display the git version on the Cloud Shell.
```

如要在電腦上設定開發環境，請安裝 Git。

在 Git 中設定使用者名稱和電子郵件地址

Git 會使用使用者名稱將提交內容與身分建立關聯。Git 使用者名稱與 GitHub 使用者名稱不同。

您可以使用 `git config` 指令，變更與 Git 提交相關聯的名稱。使用 `git config` 變更與 Git 提交相關聯的名稱，只會影響日後的提交，不會變更過去提交所用的名稱。

為電腦上的每個存放區設定 Git 使用者名稱

Mac/Linux：開啟終端機

Windows：開啟 Git Bash

設定 Git 使用者名稱：

```
$ git config --global user.name "Your Name"
$ git config --global user.email "email@example.com"
```

確認您已正確設定 Git 使用者名稱：

```
$ git config --list
> Your Name
> Your Email
```

您已成功設定 Git，現在應該可以分叉、建立及複製分支。在本程式碼研究室中，我們將大量使用 Git。

3. 為 CFT 的 GCS 存放區建立分支

建立 CFT 存放區分支

您已在上一個步驟中，在本機或 Cloud Shell 中設定 Git。現在請分叉 Google Cloud Storage CFT 存放區，開始貢獻內容。

分支是存放區的副本。您可以任意實驗變更，不會影響原始專案。

最常見的用途是提議變更他人的專案，或是以他人的專案為起點，發想自己的點子。

舉例來說，您可以透過 Fork 提議修正錯誤。如要修正錯誤，請採取下列做法：

- 為存放區建立分支。
- 修正問題。
- 向專案擁有者提交提取要求。

複製 CFT 存放區的步驟：

1. 開啟網路瀏覽器並前往 [terraform-google-modules/terraform-google-cloud-storage](https://github.com/terraform-google-modules/terraform-google-cloud-storage) 存放區。我們會在整個程式碼研究室中使用這個存放區。
2. 按一下頁面右上角的 [Fork] (分支)。



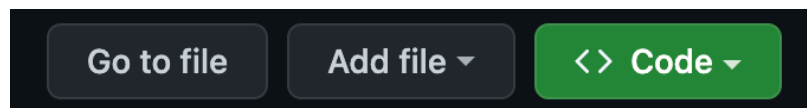
3. 系統會顯示要建立分支的位置選項，請選擇您的設定檔，然後系統就會建立分支。

在本機複製分支

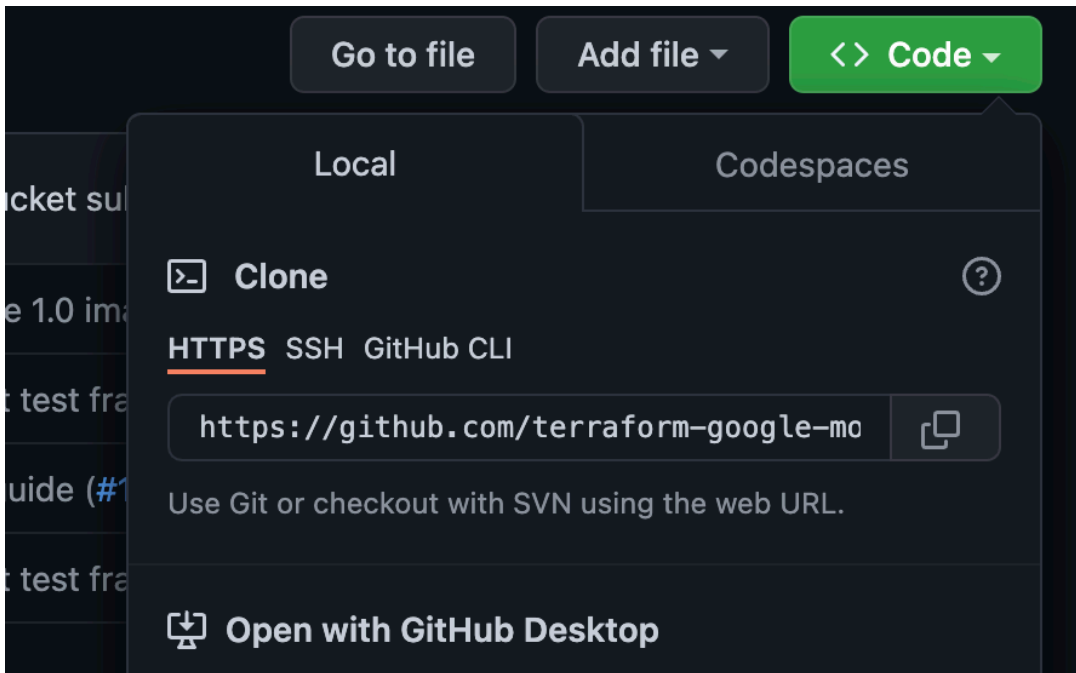
您建立的 Fork 是 GCS 模組存放區的副本。現在請將這個存放區複製到本機環境，以便新增功能。

複製分支的步驟：

1. 開啟網路瀏覽器，然後前往 [terraform-google-modules/terraform-google-cloud-storage](https://github.com/terraform-google-modules/terraform-google-cloud-storage) 上的分叉。
2. 按一下右上角的「程式碼」按鈕。



3. 按一下「Code」按鈕後，點選「Copy」圖示即可複製 Fork 的網址。您將使用這個網址，將分支複製到本機環境。



4. 前往 VSCode 或電腦中的終端機，然後複製分叉。

```
$ git clone <url>
# This command will clone your fork locally.
# Paste the copied URL from the previous step.
```

5. 您已在本機複製分支，現在應該進入存放區，從分支建立新分支，並對臨時分支進程式碼變更。

按照慣例，您可以將分支版本命名如下：

- 如要提出功能要求，請按一下 `feature/feature-name`。
- 如要瞭解內部更新，請參閱 `internal/change-name`
- 修正錯誤：`bugfix/issue-name`

由於您要新增功能，因此可以將臨時分支命名為 `feature/silly_label`

```
$ cd terraform-google-cloud-storage
# This command takes you into the cloned directory on your local machine.

$ git branch
# This command tells your current branch
# When you run this for the first time after you have cloned, your
# output should say "master", that is your fork.

$ git checkout -b feature/silly_label
# This command creates a new branch on your fork and switches your
# branch to the newly created branch.

$ git branch
# This command will confirm your current branch to be "feature/silly_label"
```

您現在已完成所有設定，可以開始使用 Cloud Foundation Toolkit 了！

步驟 4 · 約 5 分鐘

4. 建立測試環境

標準 CFT 開發程序是以使用獨立測試專案進行測試為基礎。本步驟會引導您透過服務帳戶，建立測試專案 (以標準設定為準)。

0. 安裝 Docker Engine

如果您要將電腦用於開發用途，請安裝 [Docker Engine](#)。

注意：如果您使用 Cloud Shell，則不需要安裝 Docker，因為 Cloud Shell 已內建 Docker。

1. 安裝 Google Cloud SDK

如果您使用 GCP [Cloud Shell](#)，則不需要安裝 [Google Cloud SDK](#)。

前往 [Google Cloud SDK](#)，下載適用於您平台的互動式安裝程式。

2. 設定

如要建立測試環境，您需要 Google Cloud 機構、測試資料夾和帳單帳戶。這些值必須透過環境變數設定：

```
export TF_VAR_org_id="your_org_id"
export TF_VAR_folder_id="your_folder_id"
export TF_VAR_billing_account="your_billing_account_id"
```

3. 設定服務帳戶

建立測試環境前，您需要將服務帳戶金鑰下載至測試環境。這個服務帳戶需要「專案建立者」、「帳單帳戶使用者」和「機構檢視者」角色。這些步驟可協助您建立新的服務帳戶，但您也可以重複使用現有帳戶。

3.1 建立或選取 Seed GCP 專案

建立服務帳戶前，請先選取要代管該帳戶的專案。您也可以建立新專案。

```
gcloud config set core/project YOUR_PROJECT_ID
```

3.2 啟用 Google Cloud API

在種子專案中啟用下列 Google Cloud API：

```
gcloud services enable cloudresourcemanager.googleapis.com
gcloud services enable iam.googleapis.com
gcloud services enable cloudbilling.googleapis.com
```

3.3 建立服務帳戶

建立新的服務帳戶來管理測試環境：

```
# Creating a service account for CFT.
gcloud iam service-accounts create cft-onboarding \
  --description="CFT Onboarding Terraform Service Account" \
  --display-name="CFT Onboarding"

# Assign SERVICE_ACCOUNT environment variable for later steps
export SERVICE_ACCOUNT=cft-onboarding@$(gcloud config get-value
core/project).iam.gserviceaccount.com
```

確認服務帳戶已建立：

```
gcloud iam service-accounts list --filter="EMAIL=${SERVICE_ACCOUNT}"
```

3.4 將專案建立者、帳單帳戶使用者和機構檢視者角色授予服務帳戶：

```
gcloud resource-manager folders add-iam-policy-binding ${TF_VAR_folder_id} \
  --member="serviceAccount:${SERVICE_ACCOUNT}" \
  --role="roles/resourcemanager.projectCreator"
gcloud organizations add-iam-policy-binding ${TF_VAR_org_id} \
  --member="serviceAccount:${SERVICE_ACCOUNT}" \
  --role="roles/billing.user"
gcloud beta billing accounts add-iam-policy-binding ${TF_VAR_billing_account} \
  --member="serviceAccount:${SERVICE_ACCOUNT}" \
  --role="roles/billing.user"
gcloud organizations add-iam-policy-binding ${TF_VAR_org_id} \
  --member="serviceAccount:${SERVICE_ACCOUNT}" \
  --role="roles/resourcemanager.organizationViewer"
```

您現在擁有服務帳戶，可用於管理測試環境。

4. 準備 Terraform 憑證

如要建立測試環境，您必須將服務帳戶金鑰下載到 Shell 中。

4.1 服務帳戶金鑰

建立並下載 Terraform 的服務帳戶金鑰

```
gcloud iam service-accounts keys create cft.json --iam-account=${SERVICE_ACCOUNT}
```

4.2 設定 Terraform 憑證

使用環境變數 `SERVICE_ACCOUNT_JSON` 將金鑰提供給 Terraform，並將值設為服務帳戶金鑰的內容。

```
export SERVICE_ACCOUNT_JSON=$(cat cft.json)
```

將憑證資訊儲存在環境變數中後，請移除金鑰檔案。之後如有需要，您可以使用上述相同指令重新建立金鑰。

```
rm -rf cft.json
```

5. 建立 Terraform 部署作業的測試專案

一切準備就緒後，您可以使用單一指令建立測試專案。從 terraform-google-cloud-storage 目錄根層級執行這項指令：

```
make docker_test_prepare
```

執行 `make docker_test_prepare` 時，您會看到下列輸出內容，最後會收到已建立的測試 `project_id`，您將使用新功能部署及測試 Cloud Storage 模組。如果連結帳單帳戶時發生問題，請參閱[疑難排解步驟](#)。

```
macbookpro3:terraform-google-cloud-storage user$ make docker_test_prepare
docker run --rm -it \
    -e SERVICE_ACCOUNT_JSON \
    -e TF_VAR_org_id \
    -e TF_VAR_folder_id \
    -e TF_VAR_billing_account \
    -v /Users/cft/terraform-google-cloud-storage:/workspace \
    gcr.io/cloud-foundation-cicd/cft/developer-tools:0.8.0 \
    /usr/local/bin/execute_with_credentials.sh prepare_environment
```

```
Activated service account credentials for: [cft-
onboarding@<project_id>.iam.gserviceaccount.com]
Activated service account credentials for: [cft-
onboarding@<project_id>.iam.gserviceaccount.com]
Initializing modules...
```

```
Initializing the backend...
```

```
Initializing provider plugins...
```

The following providers do not have any version constraints in configuration, so the latest version was installed.

To prevent automatic upgrades to new major versions that may contain breaking changes, it is recommended to add version = "... constraints to the corresponding provider blocks in configuration, with the constraint strings suggested below.

```
* provider.google-beta: version = "~> 3.9"
* provider.null: version = "~> 2.1"
* provider.random: version = "~> 2.2"
```

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see any changes that are required for your infrastructure. All Terraform commands should now work.

If you ever set or change modules or backend configuration for Terraform, rerun this command to reinitialize your working directory. If you forget, other commands will detect it and remind you to do so if necessary.

```
module.project.module.project-factory.null_resource.preconditions: Refreshing state...
[id=8723188031607443970]
module.project.module.project-factory.null_resource.shared_vpc_subnet_invalid_name[0]:
Refreshing state... [id=5109975723938185892]
module.project.module.gsuite_group.data.google_organization.org[0]: Refreshing state...
module.project.module.project-factory.random_id.random_project_id_suffix: Refreshing state...
[id=rnk]
module.project.module.project-factory.google_project.main: Refreshing state... [id=<project-
id>]
module.project.module.project-factory.google_project_service.project_services[0]: Refreshing
state... [id=<project-id>/storage-api.googleapis.com]
module.project.module.project-factory.google_project_service.project_services[1]: Refreshing
```

```
state... [id=<project-id>/cloudresourcemanager.googleapis.com]
module.project.module.project-factory.google_project_service.project_services[2]: Refreshing
state... [id=<project-id>/compute.googleapis.com]
module.project.module.project-factory.data.null_data_source.default_service_account:
Refreshing state...
module.project.module.project-factory.google_service_account.default_service_account:
Refreshing state... [id=projects/ci-cloud-storage-ae79/serviceAccounts/project-service-
account@<project-id>.iam.gserv
iceaccount.com]
module.project.module.project-factory.google_project_service.project_services[3]: Refreshing
state... [id=<project-id>/serviceusage.googleapis.com]
module.project.module.project-
factory.null_resource.delete_default_compute_service_account[0]: Refreshing state...
[id=3576396874950891283]
google_service_account.int_test: Refreshing state... [id=projects/<project-
id>/serviceAccounts/cft-onboarding@<project-id>.iam.gserviceaccount.com]
google_service_account_key.int_test: Refreshing state... [id=projects/<project-
id>/serviceAccounts/cft-onboarding@<project-
id>.iam.gserviceaccount.com/keys/351009ale011e88049ab2097994d1c627a61
6961]
google_project_iam_member.int_test[1]: Refreshing state... [id=<project-
id>/roles/iam.serviceAccountUser/serviceaccount:cft-onboarding@<project-
id>.iam.gserviceaccount.com]
google_project_iam_member.int_test[0]: Refreshing state... [id=<project-
id>/roles/storage.admin/serviceaccount:cft-onboarding@<project-id>.iam.gserviceaccount.com]

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

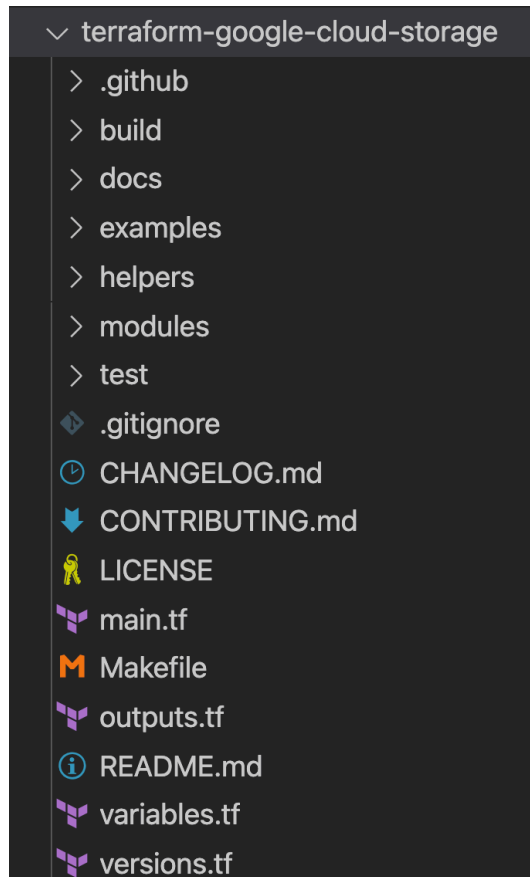
project_id = <test-project-id>
sa_key = <sensitive>
Found test/setup/make_source.sh. Using it for additional explicit environment configuration.
```

您現在已建立測試專案，如控制台輸出內容所示，該專案會由 `project_id` 參照。開發和測試環境已設定完成。

5. 在 CFT 模組中新增功能

現在開發和測試環境已設定完成，接下來請開始將「silly_label」功能新增至 google-cloud-storage CFT 模組。

請確認您位於 terraform-google-cloud-storage 中，並開啟 main.tf 檔案，如下方資料夾結構所示。



由於「silly_label」是標籤，因此您會在 main.tf 的「labels」變數中，於第 27 行新增功能，如下所示：

terraform-google-cloud-storage/main.tf

```
resource "google_storage_bucket" "buckets" {  
  <...>  
  storage_class = var.storage_class  
  // CODELAB:Add silly label in labels variable  
  labels        = merge(var.labels, { name = replace("${local.prefix}${lower(each.value)}",  
  ".", "-") }, { "silly" = var.silly_label })  
  force_destroy = lookup(  
    <...>  
  )  
}
```

現在，您要在 variables.tf 中新增 silly_label 變數，如上述資料夾結構所示。

複製並貼上下列程式碼，然後加入 variables.tf 的第 31 行，並確保您在新增的變數區塊上方和下方都有換行字元。

terraform-google-cloud-storage/variables.tf

```
variable "names" {
  description = "Bucket name suffixes."
  type       = list(string)
}

// CODELAB: Add "silly_label" variable to variables.tf between "names" and "location"
variable "silly_label" {
  description = "Sample label for bucket."
  type       = string
}

variable "location" {
  description = "Bucket location."
  default     = "EU"
}
```

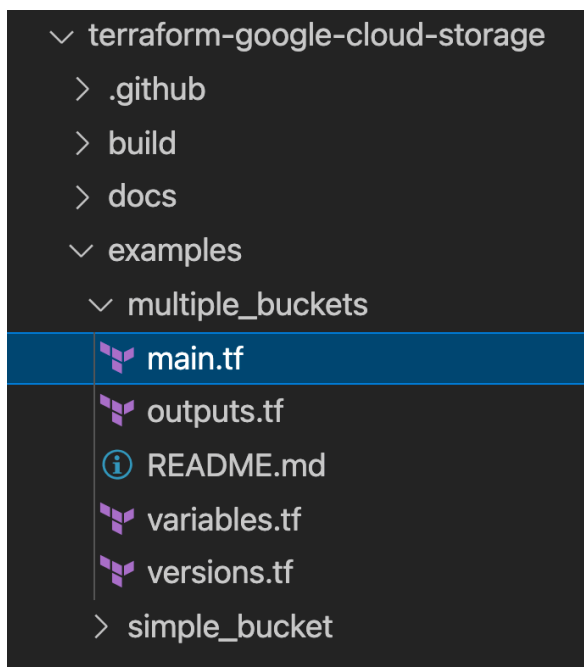
6. 在儲存空間 bucket 範例中新增功能

您已將功能新增至模組的 main.tf，現在要透過範例測試新增的功能。

「silly_label」必須新增至 examples/multiple-buckets/main.tf

這個範例會在下一個步驟中用於執行整合測試。

將下列 silly_label 變數行複製並貼到 terraform-google-cloud-storage/examples/multiple-buckets/ 中 main.tf 的第 27 行，如資料夾結構所示：



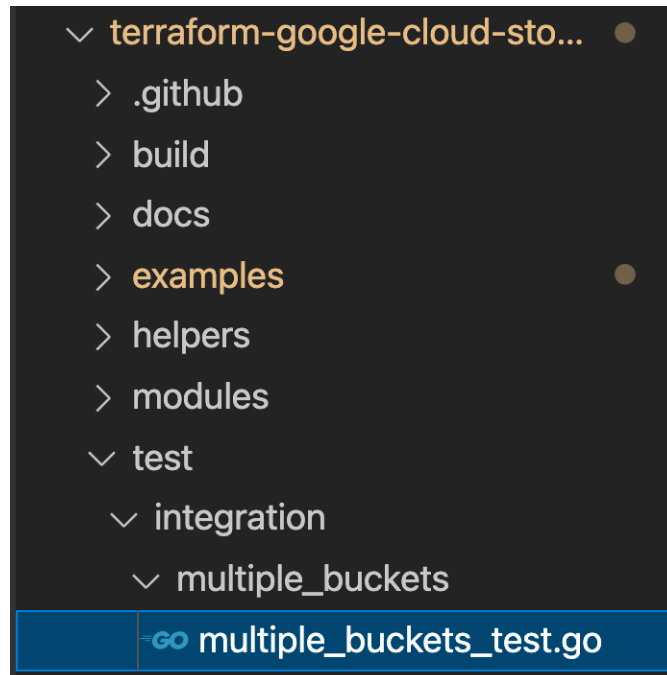
terraform-google-cloud-storage/examples/multiple-buckets/main.tf

```
module "cloud_storage" {  
  <...>  
  // CODELAB: Add "silly_label" as an example to main.tf.  
  silly_label      = "awesome"  
  
  <...>  
}
```


7. 更新藍圖測試以檢查功能

您已將功能新增至模組的 `main.tf`，然後將功能新增至 `multiple_buckets` 範例。現在，您需要透過以 Golang 撰寫的藍圖整合測試，測試這項功能。

您會在以下資料夾結構中的 `multiple_buckets_test.go` 檔案中新增測試：



您在透過 `multiple_buckets` 模組建立的所有 buckets 上新增了「`silly_label`」，現在需要編寫測試來測試新功能。

在下列程式碼中，您會透過 `gcloud alpha storage` 指令取得每個 bucket 的標籤，然後檢查指令傳回的輸出內容。

[test/integration/multiple_buckets/multiple_buckets_test.go](#)

```
func TestMultipleBuckets(t *testing.T) {
    <..>
    op := gcloud.Run(t, fmt.Sprintf("alpha storage ls --buckets gs://%s", bucketName),
        gcloudArgs).Array()[0]

    // verify silly label on each bucket
    assert.Equal("awesome", op.Get("metadata.labels.silly").String(), "should have silly label
        set to awesome")

    // verify lifecycle rules
    ...
}
```

8. 在 CFT 中執行整合測試

整合測試

整合測試用於驗證根模組、子模組和範例的行為。新增、變更及修正內容時，請一併提供測試。

整合測試是使用 [blueprint 測試架構](#) 編寫，並使用 [CFT CLI](#) 執行。為方便起見，這些工具會封裝在 Docker 映像檔中。

這類測試的一般策略是驗證 [範例模組](#) 的行為，確保根模組、子模組和範例模組的功能都正確無誤。

在互動式執行中，您會透過多個指令執行每個步驟。

1. 執行 `make docker_run`，以互動模式啟動測試 Docker 容器。

Make 是一種建構自動化工具，可讀取名為 [Makefile](#) 的檔案，從原始碼自動建構可執行程式和程式庫，這些檔案會指定如何衍生目標程式。變更檔案時，Docker 容器必須自動更新。

執行 `make docker_run` 時，您會在 Docker 容器中建立工作區，並啟用服務帳戶的憑證。後續步驟會使用這個工作區執行測試。

終端機中會顯示下列輸出內容：

```
Activated service account credentials for: [cft@<PROJECT_ID>.iam.gserviceaccount.com]
```

2. 執行 `module-swapper -registry-prefix=terraform-google-modules`，調整範例 `main.tf` 檔案，從本機檔案匯入模組，而非已發布的模組。

終端機應會顯示類似以下的輸出內容：

```
[root@<CONTAINER_ID> workspace]# module-swapper -registry-prefix=terraform-google-modules
2025/08/04 19:26:29 Module name set from remote to cloud-storage
2025/08/04 19:26:29 Modifications made to file /workspace/examples/multiple_buckets/main.tf
2025/08/04 19:26:29 --- Original
+++ Modified
@@ -21,7 +21,7 @@
 }

 module "cloud_storage" {
- source = "terraform-google-modules/cloud-storage/google"
+ source = "../.."
   # [restore-marker]   version = "~> 10.0"

   project_id = var.project_id
```

3. 執行 `cft test list`，列出工作區中的所有藍圖測試。

終端機中會顯示下列輸出內容：

```
[root@CONTAINER_ID workspace]# cft test list
```

NAME	CONFIG	LOCATION
------	--------	----------

TestAll/examples/simple_bucket	examples/simple_bucket	
test/integration/discover_test.go		
TestMultipleBuckets	examples/multiple_buckets	
test/integration/multiple_buckets/multiple_buckets_test.go		

4. 執行 `cft test run <EXAMPLE_NAME> --stage init` 初始化範例。在本例中，如要初始化 `TestMultipleBuckets` 測試執行作業，請使用 `cft test run TestMultipleBuckets --stage init`。此外，您也可以執行測試時使用 `--verbose` 旗標取得額外資訊。

這個初始化階段會初始化 Terraform 範例。

終端機中會顯示下列輸出內容。

```
[root@<CONTAINER_ID> workspace]# cft test run TestMultipleBuckets --stage init --verbose
INFO[02-09|08:24:31] using test-dir: test/integration
...
TestMultipleBuckets 2022-02-09T08:24:35Z command.go:179: Terraform has been successfully
initialized!
...
TestMultipleBuckets 2022-02-09T08:24:35Z command.go:100: Running command terraform with args
[validate]
TestMultipleBuckets 2022-02-09T08:24:36Z command.go:179: Success! The configuration is valid.
...
--- PASS: TestMultipleBuckets (4.05s)
```

注意：記錄中可能會顯示類似 `Skipping fixtures discovery` 的警告。這些是偵錯警告，可以忽略。如果發生錯誤，測試就會失敗。

5. 執行 `cft test run <EXAMPLE_NAME> --stage apply`，套用範例模組。

這個步驟會將上一個階段初始化的範例套用至您稍早在程式碼研究室中建立的 GCP 專案。

終端機中會顯示下列輸出內容。

```
[root@<CONTAINER_ID> workspace]# cft test run TestMultipleBuckets --stage apply --verbose
INFO[02-09|08:28:11] using test-dir: test/integration
...
TestMultipleBuckets 2022-02-09T08:28:19Z command.go:179: Apply complete! Resources: 6 added,
0 changed, 0 destroyed.
TestMultipleBuckets 2022-02-09T08:28:19Z command.go:179:
TestMultipleBuckets 2022-02-09T08:28:19Z command.go:179: Outputs:
TestMultipleBuckets 2022-02-09T08:28:19Z command.go:179:
TestMultipleBuckets 2022-02-09T08:28:19Z command.go:179: names = {
TestMultipleBuckets 2022-02-09T08:28:19Z command.go:179:   "one" = "multiple-buckets-erpl-eu-
one"
...
--- PASS: TestMultipleBuckets (6.51s)
PASS
ok      github.com/terraform-google-modules/terraform-google-cloud-
storage/test/integration/multiple_buckets    6.548s
```

注意：如果記錄中顯示 `Request violates constraint`

`'constraints/storage.uniformBucketLevelAccess'`，表示專案禁止將 `uniform_bucket_level_access` 設為 `false`。在本程式碼研究室中，您可以將 `cloud_storage.bucket_policy_only["two"]` 設為 `true`，而非 `false`，並相應變更 Go 測試，藉此解決這個問題。

6. 執行 `cft test run <EXAMPLE_NAME> --stage verify`，確認範例已建立預期的基礎架構。

這個步驟會在 `TestMultipleBuckets` 中執行驗證函式。通常，驗證作業是透過執行 `gcloud` 指令來完成，以擷取資源目前狀態的 JSON 輸出內容，並確認目前狀態與範例中宣告的狀態一致。

如果發生任何錯誤，您會看到測試指令預期收到的內容，以及實際收到的內容。

終端機中會顯示下列輸出內容。

```
[root@<CONTAINER_ID> workspace]# cft test run TestMultipleBuckets --stage verify --verbose
INFO[02-09|08:30:19] using test-dir: test/integration
...
TestMultipleBuckets 2022-02-09T08:30:27Z command.go:100: Running command terraform with args
[output -no-color -json names_list]
TestMultipleBuckets 2022-02-09T08:30:27Z command.go:179: ["multiple-buckets-erpl-eu-
one", "multiple-buckets-erpl-eu-two"]
TestMultipleBuckets 2022-02-09T08:30:27Z command.go:100: Running command gcloud with args
[alpha storage ls --buckets gs://multiple-buckets-erpl-eu-one --project ci-cloud-storage-8ce9
--json]
TestMultipleBuckets 2022-02-09T08:30:28Z command.go:179: [
TestMultipleBuckets 2022-02-09T08:30:28Z command.go:179: {
TestMultipleBuckets 2022-02-09T08:30:28Z command.go:179:   "url": "gs://multiple-buckets-
erpl-eu-one/",
...
TestMultipleBuckets 2022-02-09T08:30:33Z command.go:179: ]
2022/02/09 08:30:33 RUN_STAGE env var set to verify
2022/02/09 08:30:33 Skipping stage teardown
--- PASS: TestMultipleBuckets (12.32s)
PASS
ok      github.com/terraform-google-modules/terraform-google-cloud-
storage/test/integration/multiple_buckets    12.359s
```

7. 執行 `cft test run <EXAMPLE_NAME> --stage teardown` 拆除範例。

這個步驟會刪除您在上述步驟中建立的基礎架構。這個步驟也會一併銷毀專案中建立的 GCS 值區，以及您新增至 GCS 模組的標籤。

您可以在終端機中查看下列輸出內容。

```
[root@<CONTAINER_ID> workspace]# cft test run TestMultipleBuckets --stage teardown --verbose
INFO[02-09|08:36:02] using test-dir: test/integration
...
TestMultipleBuckets 2022-02-09T08:36:06Z command.go:100: Running command terraform with args
[destroy -auto-approve -input=false -lock=false]
TestMultipleBuckets 2022-02-09T08:36:07Z command.go:179:
module.cloud_storage.random_id.bucket_suffix: Refreshing state... [id=mNA]
TestMultipleBuckets 2022-02-09T08:36:07Z command.go:179: random_string.prefix: Refreshing
state... [id=erpl]
TestMultipleBuckets 2022-02-09T08:36:08Z command.go:179:
module.cloud_storage.google_storage_bucket.buckets["two"]: Refreshing state... [id=multiple-
buckets-erpl-eu-two]
...
TestMultipleBuckets 2022-02-09T08:36:10Z command.go:179: Destroy complete! Resources: 6
destroyed.
TestMultipleBuckets 2022-02-09T08:36:10Z command.go:179:
--- PASS: TestMultipleBuckets (6.62s)
PASS
ok      github.com/terraform-google-modules/terraform-google-cloud-
storage/test/integration/multiple_buckets    6.654s
```

8. 執行 `module-swapper -registry-prefix=terraform-google-modules -restore`，還原上次執行 `module-swapper` 時對範例 `main.tf` 檔案所做的調整。

```
[root@<CONTAINER_ID> workspace]# module-swapper -registry-prefix=terraform-google-modules -restore
2025/08/04 19:30:41 Module name set from remote to cloud-storage
2025/08/04 19:36:32 Modifications made to file /workspace/examples/multiple_buckets/main.tf
2025/08/04 19:36:32 --- Original
+++ Modified
@@ -21,8 +21,8 @@
 }

 module "cloud_storage" {
- source = "../.."
- version = "~> 10.0"
+ source = "terraform-google-modules/cloud-storage/google"
+ version = "~> 10.0"

 project_id = var.project_id
```

9. 執行 `exit` 即可結束測試容器。

步驟 9 · 約 10 分鐘

9. 生成輸入和輸出內容的說明文件

系統會根據根模組、子模組和範例模組的 `variables` 和 `outputs`，自動產生 README 中的「輸入」和「輸出」表格。如果模組介面有所變更，就必須重新整理這些表格。

執行作業：

```
make generate_docs
# This will generate new Inputs and Outputs tables
```

提醒：請務必執行 `generate_docs`，確保所有文件都已完成。

步驟 10 · 約 5 分鐘

10. 在 CFT 中執行 Lint 測試

Lint 是一種工具，可分析原始碼，標示程式設計錯誤、錯誤、樣式錯誤和可疑建構。

存放區中的許多檔案都可以進行 Lint 檢查或格式化，以維持品質標準。為確保 CFT 品質，您將使用 lint 測試。

執行作業：

```
make docker_test_lint
# This will run all lint tests on your repo
```

注意：請務必執行 Lint 測試，因為如果 Lint 測試未完成，Cloud Build 就會失敗。請先執行這項操作，再將程式碼提交至本機存放區。

11. 在 GitHub 上提交提取要求

注意：此時程式碼研究室已完成。如果您是 GitHub 新手，想瞭解相關流程，建議您建立提取要求。

您已在本機變更程式碼，並透過整合測試進行測試，現在要將這段程式碼發布至主要存放區。

如要在主要存放區提供程式碼，您必須將程式碼變更提交至分支版本，並推送至主要存放區。如要將程式碼新增至您在程式碼研究室開始時 Fork 的主要存放區，請先將程式碼提交至您的存放區，然後在主要存放區中提出 Pull Request (PR)。

您提出 PR 後，系統會通知存放區管理員審查提議的程式碼變更。此外，您也可以新增其他使用者做為審查人員，取得程式碼變更的意見回饋。PR 會觸發 Cloud Build，在存放區中執行測試。

程式碼審查人員會根據您的程式碼變更提供註解，並根據最佳做法和文件要求修改。管理員會檢查程式碼變更，確保程式碼符合存放區規定，並可能再次要求您進行一些變更，然後才會將程式碼合併至主要存放區。

執行下列步驟，將程式碼提交至已分支的分支，並將程式碼推送至已分支的分支：

1. 第一步是將變更的檔案新增至本機存放區。

```
$ git add main.tf
$ git add README.md
$ git add variables.tf
$ git add examples/multiple-buckets/main.tf
$ git add test/integration/multiple_buckets/multiple_buckets_test.go
# The 'git add' command adds the file in the local repository and
# stages the file for commit. To unstage a file, use git reset HEAD YOUR-FILE
```

2. 檔案現已暫存，接下來請提交變更。

```
$ git commit -m "First CFT commit"
# This will commit the staged changes and prepares them to be pushed
# to a remote repository. To remove this commit and modify the file,
# use 'git reset --soft HEAD~1' and commit and add the file again.
```

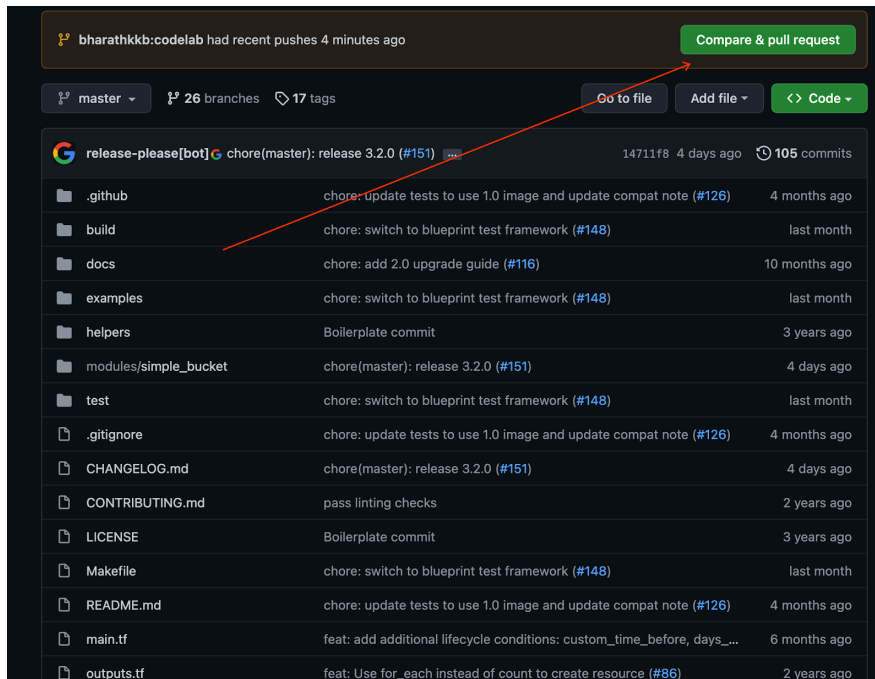
3. 將本機存放區中已提交的變更推送至 GitHub，以建立提取要求 (PR)。

```
$ git push -u origin feature/silly_label
# Pushes the changes in your local repository up to the remote
# repository you specified as the origin
```

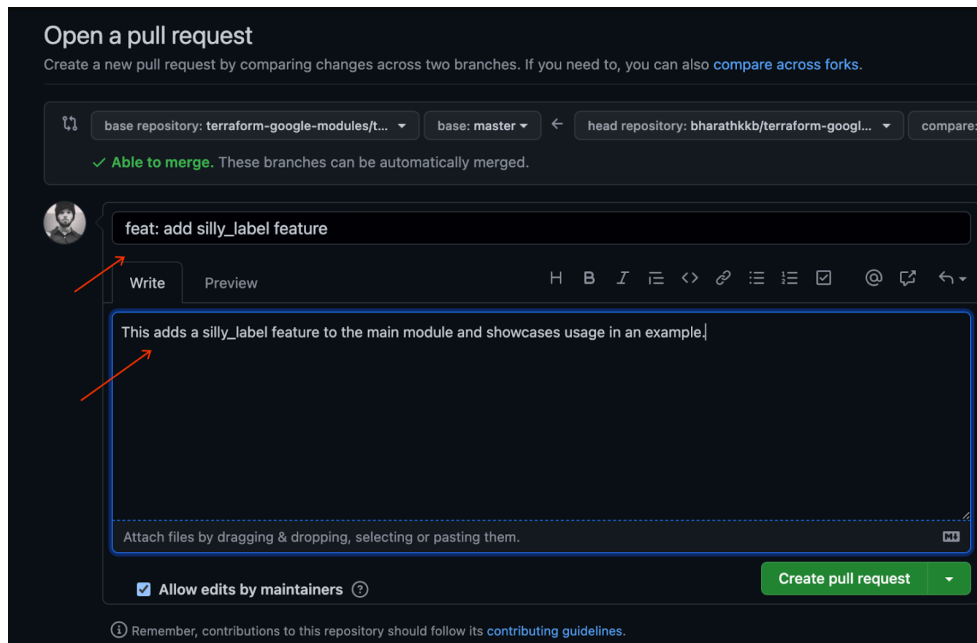
程式碼變更現在已可供提取要求使用！

執行下列步驟，在 [terraform-google-modules/terraform-google-cloud-storage](#) 存放區中建立 PR：

1. 在網路瀏覽器中，前往 [存放區](#) 的主頁面。
2. 系統會透過橫幅顯示建議，讓您從自己的 Fork 開啟 PR。按一下「比較和提取要求」。



3. 輸入提取要求的標題和說明，說明程式碼變更。請盡量具體說明，但要簡潔扼要。



4. 如要建立可供審查的提取要求，請按一下「建立提取要求」。

5. 您會看到因 PR 而觸發的 Cloud Build 觸發條件正在執行。

6. 如有任何問題，請參閱 GitHub 官方文件，瞭解如何[從分支開啟提取要求](#)。

您已成功將第一個程式碼變更推送至分支版本，並針對主要分支版本提出第一個 CFT PR！

步驟 12 · 約 0 分鐘

12. 恭喜

恭喜，您已成功將功能新增至 CFT 模組，並提交 PR 供審查！

您在 CFT 模組中新增了一項功能，透過範例在本機測試，並在將程式碼提交至 GitHub 前執行測試。最後，您提出 PR 以供審查，並最終合併至 CFT。

您現在已瞭解開始使用 Cloud Foundation Toolkit 的重要步驟。
